

Lab 6:

Supporto hardware alle procedure

Obiettivi

- Tradurre procedure da C ad assembly
- Far pratica con le "convenzioni di chiamata"
- Far pratica con l'utilizzo dello stack

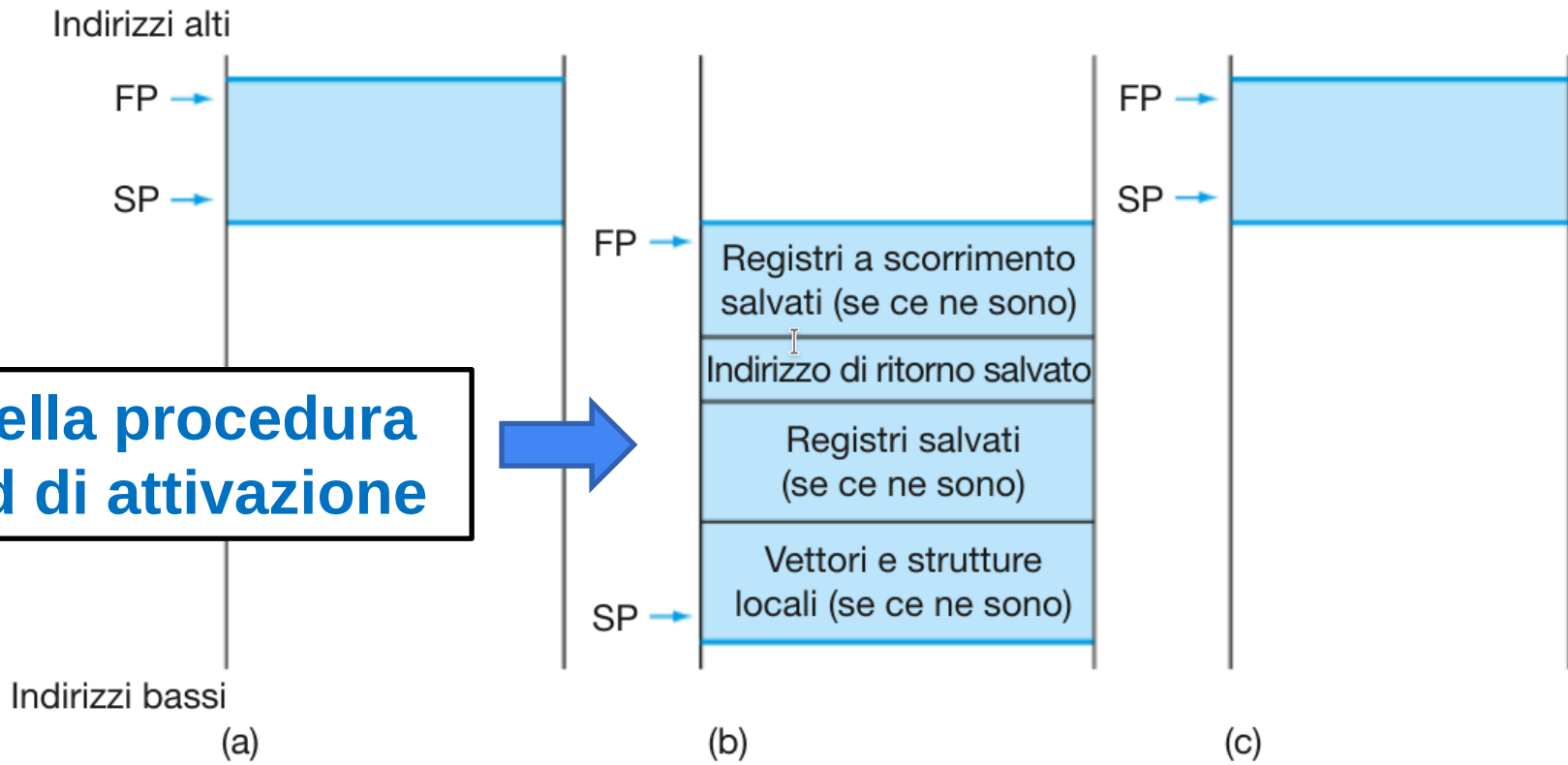
Procedura: sottoprogramma memorizzato che svolge un compito specifico basandosi sui parametri che gli vengono passati in ingresso.

Salvataggio sullo stack del contenuto dei registri

Stack Pointer (sp)

- Nel caso servano più degli 8 registri $a0-a7$ ($x10-x17$), **dobbiamo copiare i valori in memoria**
- La struttura dati utilizzata a questo fine è lo **stack**.
- **Lo stack pointer (sp)** contiene l'indirizzo della cima dello stack
- Lo stack memorizza i registri che devono essere salvati prima della chiamata alle procedure, i parametri addizionali da passare alla procedura, le variabili locali ecc.
- Il processo di trasferimento in memoria delle variabili utilizzate meno di frequente (oppure di quelle che verranno utilizzate successivamente) si chiama **register spilling** (versamento dei registri)

Stack



- Lo stack 'cresce' **da indirizzi di memoria alti verso indirizzi di memoria bassi**
- Quindi quando vengono inseriti dati nello stack il valore dello **sp diminuisce**
- **sp aumenta** quando i dati sono estratti dallo stack

Esempio (multiply)

```
int main(){  
    int a = 3;  
    int b = 4;  
    int result;  
    result = multiply(a,b);  
  
    printf("res: %d\n", result);  
    exit(0);  
}
```

chiamata di funzione



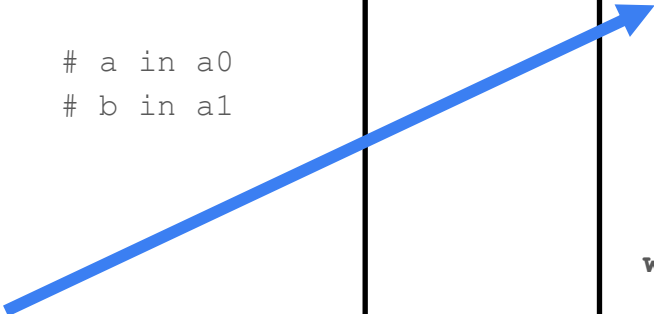
```
int multiply(int a, int b){  
    int i = 0;  
    int acc = 0;  
    while(i < b){  
        acc += a;  
        ++i;  
    }  
    return acc;  
}
```

Esempio (multiply)

```
_start:
    li a0, 3          # a in a0
    li a1, 4          # b in a1

    li s1, 10
    li t0, 13

    jal multiply
    add t1, a0, zero  # result t1
    ...
```



Semplifichiamo: multiply non usa il frame pointer (non ne ha bisogno...)

```
# a0 -> a
# a1 -> b
# return in a0
multiply:
    addi    sp, sp, -4
    sw      s1, 0(sp)
    li      s1, 0      # acc
    li      t0, 0      # i

whileloop:
    beq      t0, a1, endwhile
    add      s1, s1, a0
    addi     t0, t0, 1
    j        whileloop

endwhile:
    add      a0, s1, zero
    lw       s1, 0(sp)
    addi     sp, sp, 4
    jr       ra
```

Esempio (multiply)

```
_start:  
    li a0, 3          # a in a0  
    li a1, 4          # b in a1  
  
    li s1, 10  
    li t0, 13  
  
    jal multiply  
    add t1, a0, zero  # result t1  
    ...
```



**Il chiamante aveva impostato altri 2
registri (s1 e t0)**

```
# a0 -> a  
# a1 -> b  
# return in a0  
multiply:  
    addi    sp, sp, -4  
    sw      s1, 0(sp)  
    li      s1, 0      # acc  
    li      t0, 0      # i  
  
whileloop:  
    beq     t0, a1, endwhile  
    add     s1, s1, a0  
    addi    t0, t0, 1  
    j      whileloop  
  
endwhile:  
    add     a0, s1, zero  
    lw      s1, 0(sp)  
    addi    sp, sp, 4  
    jr     ra
```


Esempio (multiply)

```
_start:
    li a0, 3          # a in a0
    li a1, 4          # b in a1

    li s1, 10
    li t0, 13

    jal multiply
    add t1, a0, zero   # result t1
    ...
```

Cosa possiamo aspettarci per s1 e t0 dopo multiply?

```
# a0 -> a
# a1 -> b
# return in a0
```

multiply:

```
    addi    sp, sp, -4
    sw      s1, 0(sp)
    li      s1, 0      # acc
    li      t0, 0      # i
```

whileloop:

```
    beq     t0, a1, endwhile
    add     s1, s1, a0
    addi    t0, t0, 1
    j       whileloop
```

endwhile:

```
    add     a0, s1, zero
    lw      s1, 0(sp)
    addi    sp, sp, 4
    jr      ra
```

Esempio (multiply)

Address	Code	Basic	
0x00400000	0x00300513	addi x10,x0,3	2: li a0, 3
0x00400004	0x00400593	addi x11,x0,4	3: li a1, 4
0x00400008	0x00a00493	addi x9,x0,10	5: li s1, 10
0x0040000c	0x00d00293	addi x5,x0,13	6: li t0, 13
0x00400010	0x010000ef	jal x1,16	8: jal multiply



stato **prima** di aver eseguito jal multiply

- pc vale 0x00**400010**
- ra vale 0x00000000
- s1 vale 0x0000000**a**
- t0 vale 0x0000000**d**

```
# a0 -> a
# a1 -> b
# return in a0
```

multiply:

```
    addi    sp, sp, -4
    sw      s1, 0(sp)
    li      s1, 0      # acc
    li      t0, 0      # i
```

whileloop:

```
    beq     t0, a1, endwhile
    add     s1, s1, a0
    addi    t0, t0, 1
    j       whileloop
```

endwhile:

```
    add     a0, s1, zero
    lw      s1, 0(sp)
    addi    sp, sp, 4
    jr      ra
```

Esempio (multiply)

0x00400024	0x00912023	sw x9,0(x2)	20:	sw	s1, 0(sp)
0x00400028	0x00000493	addi x9,x0,0	21:	li	s1, 0 # acc
0x0040002c	0x00000293	addi x5,x0,0	22:	li	t0, 0 # i
0x00400030	0x00b28863	beq x5,x1,16	25:	beq	t0, a1, endwhile
0x00400034	0x00a484b3	add x9,x1,x10	26:	add	s1, s1, a0
0x00400038	0x00128293	addi x5,x5,1	27:	addi	t0, t0, 1
0x0040003c	0xff5ff06f	jal x0,-12	28:	j	whileloop

Multiply usa sia s1 che t0 come registri di appoggio

- pc vale 0x00400030
- ra vale 0x00400014
- s1 vale 0x00000000
- t0 vale 0x00000000

```
# a0 -> a
# a1 -> b
# return in a0
```

multiply:

```
addi    sp, sp, -4
sw      s1, 0(sp)
li      s1, 0    # acc
li      t0, 0    # i
```

whileloop:

```
beq      t0, a1, endwhile
add      s1, s1, a0
addi     t0, t0, 1
j        whileloop
```

endwhile:

```
add      a0, s1, zero
lw       s1, 0(sp)
addi     sp, sp, 4
jr       ra
```

Esempio (multiply)

0x00400024	0x00912023	sw x9,0(x2)	20:	sw	s1, 0(sp)
0x00400028	0x00000493	addi x9,x0,0	21:	li	s1, 0 # acc
0x0040002c	0x00000293	addi x5,x0,0	22:	li	t0, 0 # i
0x00400030	0x00b28863	beq x5,x11,16	25:	beq	t0, a1, endwhile
0x00400034	0x00a484b3	add x9,x9,x10	26:	add	s1, s1, a0
0x00400038	0x00128293	addi x5,x5,1	27:	addi	t0, t0, 1
0x0040003c	0xff5ff06f	jal x0,-12	28:	j	whileloop

Il chiamato deve salvare i registri s* (se usati)

```
# a0 -> a
# a1 -> b
# return in a0
```

multiply:

```
addi    sp, sp, -4
sw       s1, 0(sp)
li       s1, 0    # acc
li       t0, 0    # i
```

whileloop:

```
beq      t0, a1, endwhile
add      s1, s1, a0
addi     t0, t0, 1
j        whileloop
```

endwhile:

```
add      a0, s1, zero
lw       s1, 0(sp)
addi     sp, sp, 4
jr       ra
```

Esempio (multiply)

Address	Code	Basic	
0x00400000	0x00300513	addi x10,x0,3	2: li a0, 3
0x00400004	0x00400593	addi x11,x0,4	3: li a1, 4
0x00400008	0x00a00493	addi x9,x0,10	5: li s1, 10
0x0040000c	0x00d00293	addi x5,x0,13	6: li t0, 13
0x00400010	0x010000ef	jal x1,16	8: jal multiply
0x00400014	0x00050333	add x6,x10,x0	9: add t1, a0, zero



t0 sovrascritto

stato **dopo** aver eseguito jal multiply

- pc vale 0x00400014
- ra vale 0x00400014
- s1 vale 0x0000000a
- t0 vale 0x00000004

```
# a0 -> a
# a1 -> b
# return in a0
```

multiply:

```
    addi    sp, sp, -4
    sw      s1, 0(sp)
    li      s1, 0      # acc
    li      t0, 0      # i
```

whileloop:

```
    beq     t0, a1, endwhile
    add     s1, s1, a0
    addi    t0, t0, 1
    j       whileloop
```

endwhile:

```
    add     a0, s1, zero
    lw      s1, 0(sp)
    addi    sp, sp, 4
    jr      ra
```

Procedure annidate:

Procedure che chiamano altre procedure

```
int main(){
    int a = 3;
    int b = 4;
    int result;
    result = multiply(a,b);

    printf("res: %d\n", result);
    exit(0);
}
```

chiamata di funzione

```
int multiply(int a, int b){
    int i = 0;
    int acc = 0;
    while(i < b){
        acc = sum(a, acc);
        ++i;
    }
    return acc;
}
```

chiamata di funzione

```
int sum(int a, int b){
    return a + b;
}
```

PROBLEMA: sovrascrittura dei valori nei registri **a0-a7** e in **ra**.

- nel momento in cui iniziamo ad eseguire **multiply**, **ra** viene assegnato con un valore riferito al chiamante (il **main**, nel nostro caso). Quando **multiply** chiama **sum**, **ra** viene sovrascritto con il ritorno relativo alla procedura **multiply**...
- dobbiamo quindi salvare il primo indirizzo di ritorno (al **main**) prima di chiamare **sum**.

```
int multiply(int a, int b){  
    int i = 0;  
    int acc = 0;  
    while(i < b){  
        acc = sum(a, acc);  
        ++i;  
    }  
    return acc;  
}
```


Convenzioni di chiamata

Per evitare costose operazioni di spilling (salvataggio su stack) e di restore (ri-salvataggio da stack a registri) utilizziamo una convenzione. Dividiamo i registri in 2 categorie: **quelli preservati** nel passaggio fra chiamate di funzione, e quelli **non preservati** fra le chiamate

Convenzioni di chiamata

Nome	Utilizzo	Chi lo salva
zero	La costante 0	N.A.
ra	Indirizzo di ritorno	Chiamante
sp	Puntatore a stack	Chiamato
gp	Puntatore globale	---
tp	Puntatore a thread	---
t0-t2	Temporanei	Chiamante
s0/_fp	Salvato/puntatore a frame	Chiamato
s1	Salvato	Chiamato
a0-a1	Argomenti di funzione/valori restituiti	Chiamante
a2-a7	Argomenti di funzione	Chiamante
s2-s11	Registri salvati	Chiamato
t3-t6	Temporanei	Chiamante

- **Register Spilling:**
Trasferire variabili da registri a memoria.
- I registri sono più veloce che la memoria, quindi vogliamo **evitare il "register spilling"**
- Quando dobbiamo, usiamo lo stack per fare Register Spilling

Convenzioni di chiamata - **chiamante**

Nome	Utilizzo	Chi lo salva
zero	La costante 0	N.A.
ra	Indirizzo di ritorno	Chiamante
sp	Puntatore a stack	Chiamato
gp	Puntatore globale	---
tp	Puntatore a thread	---
t0-t2	Temporanei	Chiamante
s0_-_fp	Salvato/puntatore a frame	Chiamato
s1	Salvato	Chiamato
a0-a1	Argomenti di funzione/valori restituiti	Chiamante
a2-a7	Argomenti di funzione	Chiamante
s2-s11	Registri salvati	Chiamato
t3-t6	Temporanei	Chiamante

Sempre

Convenzioni di chiamata - **chiamante**

Nome	Utilizzo	Chi lo salva
zero	La costante 0	N.A.
ra	Indirizzo di ritorno	Chiamante
sp	Puntatore a stack	Chiamato
gp	Puntatore globale	---
tp	Puntatore a thread	---
t0-t2	Temporanei	Chiamante
s0_-_fp	Salvato/puntatore a frame	Chiamato
s1	Salvato	Chiamato
a0-a1	Argomenti di funzione/valori restituiti	Chiamante
a2-a7	Argomenti di funzione	Chiamante
s2-s11	Registri salvati	Chiamato
t3-t6	Temporanei	Chiamante

**Se servono
al chiamante**

**Se servono
al chiamante**

Convenzioni di chiamata - chiamante

Nome	Utilizzo	Chi lo salva
zero	La costante 0	N.A.
ra	Indirizzo di ritorno	Chiamante
sp	Puntatore a stack	Chiamato
gp	Puntatore globale	---
tp	Puntatore a thread	---
t0-t2	Temporanei	Chiamante
s0/_fp	Salvato/puntatore a frame	Chiamato
s1	Salvato	Chiamato
a0-a1	Argomenti di funzione/valori restituiti	Chiamante
a2-a7	Argomenti di funzione	Chiamante
s2-s11	Registri salvati	Chiamato
t3-t6	Temporanei	Chiamante

**Se ci sono
parametri e
valori di ritorno**

Convenzioni di chiamata - **chiamato**

Nome	Utilizzo	Chi lo salva
zero	La costante 0	N.A.
ra	Indirizzo di ritorno	Chiamante
sp	Puntatore a stack	Chiamato
gp	Puntatore globale	---
tp	Puntatore a thread	---
t0-t2	Temporanei	Chiamante
s0/_fp	Salvato/puntatore a frame	Chiamato
s1	Salvato	Chiamato
a0-a1	Argomenti di funzione/valori restituiti	Chiamante
a2-a7	Argomenti di funzione	Chiamante
s2-s11	Registri salvati	Chiamato
t3-t6	Temporanei	Chiamante

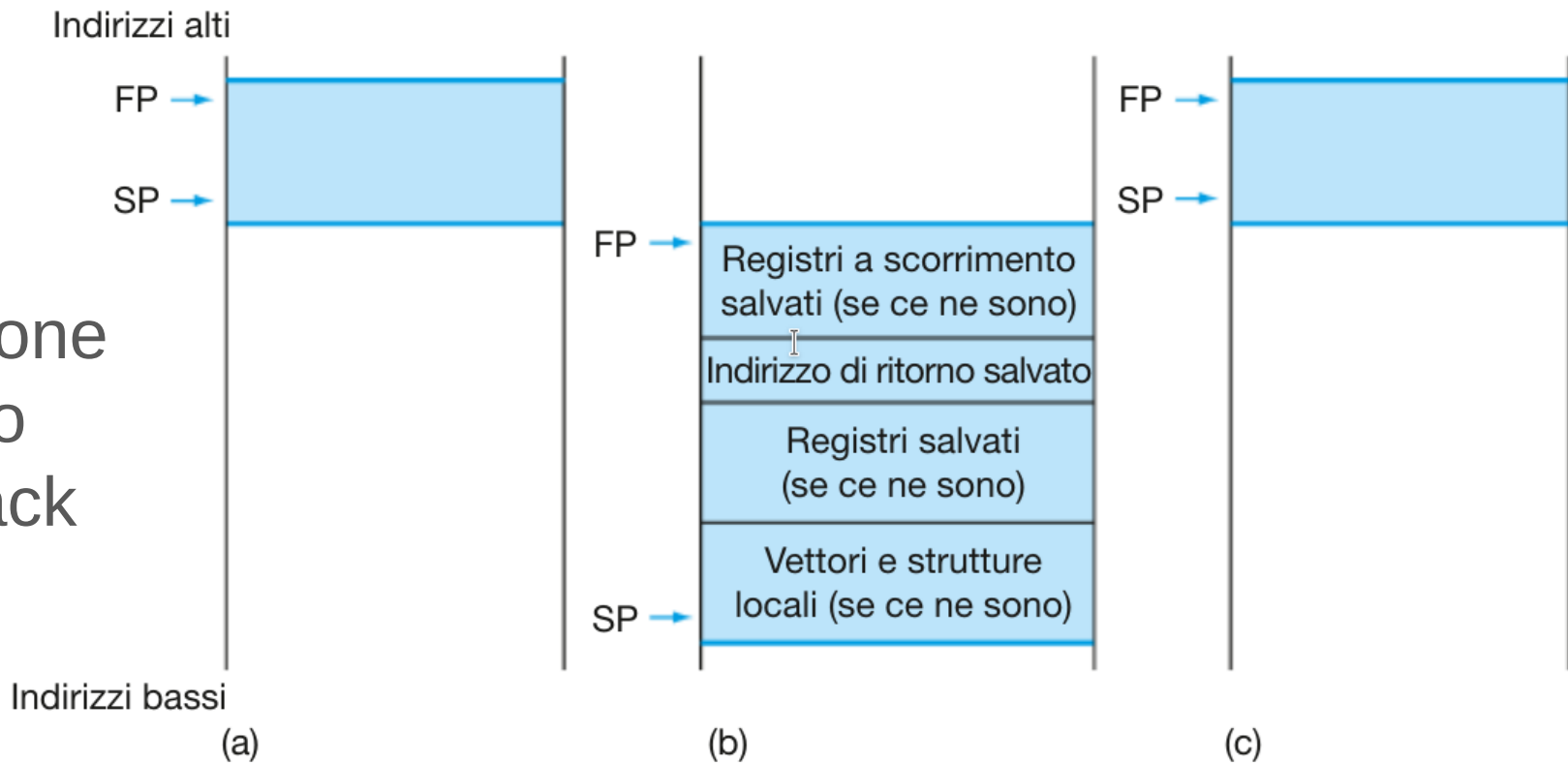
**add/sub
sempre lo stesso
numero di byte**

Convenzioni di chiamata - **chiamato**

Nome	Utilizzo	Chi lo salva
zero	La costante 0	N.A.
ra	Indirizzo di ritorno	Chiamante
sp	Puntatore a stack	Chiamato
gp	Puntatore globale	---
tp	Puntatore a thread	---
t0-t2	Temporanei	Chiamante
s0_-_fp	Salvato/puntatore a frame	Chiamato
s1	Salvato	Chiamato
a0-a1	Argomenti di funzione/valori restituiti	Chiamante
a2-a7	Argomenti di funzione	Chiamante
s2-s11	Registri salvati	Chiamato
t3-t6	Temporanei	Chiamante

**quando vengono
usati**

Allocazione di spazio sullo stack



- Se lo stack **non contiene variabili locali** alla procedura, il compilatore risparmia tempo di esecuzione **evitando di impostare e ripristinare il frame**.
- Quando viene utilizzato, **FP** viene inizializzato con **l'indirizzo** che ha **SP** all'atto della chiamata della procedura e **SP** viene ripristinato al termine della procedura utilizzando il valore di **FP**

Esercizio 1 - MCD(a,b)

Scrivere una procedura RISC-V per il calcolo del **massimo comune divisore** di due numeri interi positivi **a** e **b**. A tale scopo, implementare l'algoritmo di Euclide come procedura **MCD(a,b)** da richiamare nel main. L'algoritmo di Euclide in pseudo-codice è il seguente:

```
int MCD(int a, int b) {  
    while (a != b)  
        if (a > b)  
            a = a - b;  
        else  
            b = b - a;  
    return a;  
}
```

```
void main() {  
    int a = 24;  
    int b = 30;  
    int result;  
  
    result = MCD(a,b);  
    printf("%d\n", result);  
}
```

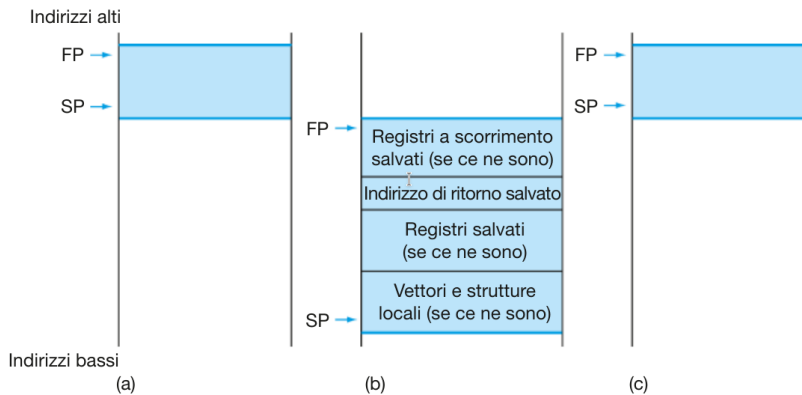
- Quante istruzioni RISC-V sono necessarie per implementare la funzione?
- Quante istruzioni RISC-V verranno eseguite per completare la funzione quando a=24, b=30?

Esercizio 1 - MCD(a,b)

```
# a0 -> a  
# a1 -> b  
# return MCD su a0
```

mcd:

ret



```
int MCD(int a, int b) {  
    while (a != b)  
        if (a > b)  
            a = a - b;  
        else  
            b = b - a;  
    return a;  
}
```

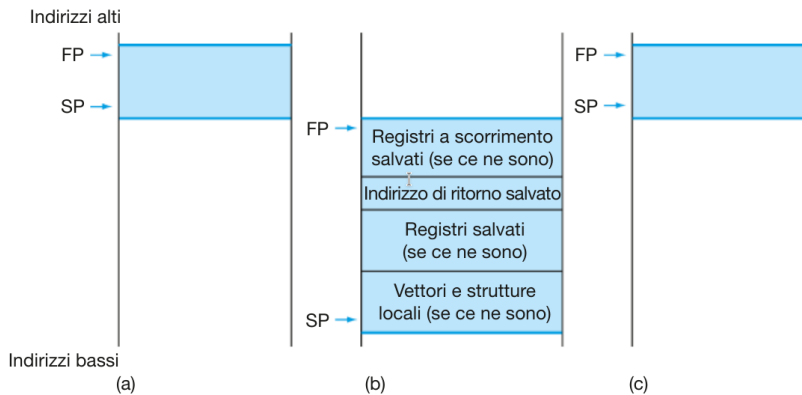
Esercizio 1 - MCD(a,b)

```
# a0 -> a
# a1 -> b
# return MCD su a0
```

mcd:

```
addi    sp, sp, -4
sw      fp, 0(sp)
```

```
lw      fp, 0(sp)
addi    sp, sp, 4
ret
```



```
int MCD(int a, int b) {
    while (a != b)
        if (a > b)
            a = a - b;
        else
            b = b - a;
    return a;
}
```

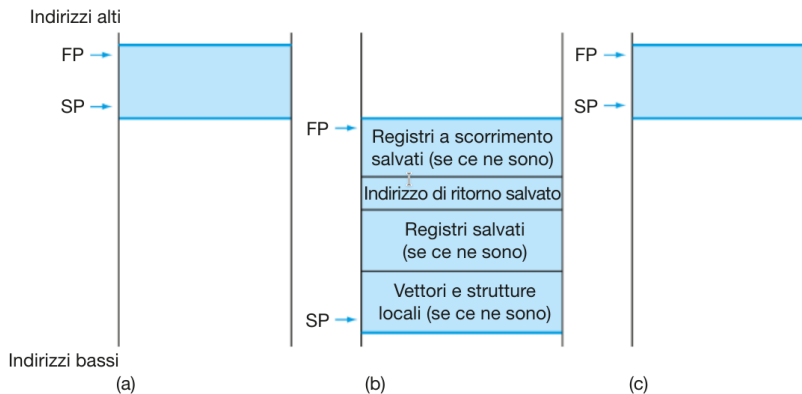
Esercizio 1 - MCD(a,b)

```
# a0 -> a  
# a1 -> b  
# return MCD su a0
```

mcd:

```
addi    sp, sp, -4  
sw      fp, 0(sp)
```

```
lw      fp, 0(sp)  
addi    sp, sp, 4  
ret
```



ottimizziamo: mcd non usa il frame pointer (non ne ha bisogno...)

```
int MCD(int a, int b) {  
    while (a != b)  
        if (a > b)  
            a = a - b;  
        else  
            b = b - a;  
    return a;  
}
```

Esercizio 1 - MCD(a,b)

```
# a0 -> a
# a1 -> b
# return MCD su a0
```

mcd:

mcd_while:

```
    beq    a0, a1, mcd_end
    bge    a1, a0, mcd_else
    sub    a0, a0, a1
    j      mcd_while
```

mcd_else:

```
    sub    a1, a1, a0
    j      mcd_while
```

mcd_end:

```
    ret
```

```
int MCD(int a, int b) {
    while (a != b)
        if (a > b)
            a = a - b;
        else
            b = b - a;
    return a;
}
```

Esercizio 1 - MCD(a,b)

```
# a0 -> a
# a1 -> b
# return MCD su a0
```

mcd:

mcd_while:

```
    beq    a0, a1, mcd_end
    bge    a1, a0, mcd_else
    sub    a0, a0, a1
    j      mcd_while
```

mcd_else:

```
    sub    a1, a1, a0
    j      mcd_while
```

mcd_end:

ret

```
void main() {
    int a = 24;
    int b = 30;
    int result;

    result = MCD(a,b);
    printf("%d\n", result);
}
```

_start:

```
    li     a0, 24
    li     a1, 30
    jal    ra, mcd
    mv     t0, a0
```

print:

```
    addi   a0, t0, 0
    li     a7, 1
    ecall
```

Esercizio 2 - MCM(a,b)

Scrivere una procedura RISC-V per il calcolo del **minimo comune multiplo** di due numeri interi positivi **a** e **b**, **MCM(a,b)**, da richiamare nel main, utilizzando la seguente relazione:

$$\text{MCM}(a,b) = (a*b) / \text{MCD}(a,b)$$

- È possibile realizzare la funzione senza riversare i registri in memoria?
- Quante istruzioni RISC-V sono necessarie per implementare la procedura?
- Quante istruzioni RISC-V verranno eseguite per completare la procedura quando $a=12$, $b=9$?

Esercizio 2 - MCM(a,b)

```
# Procedure MCM(a,b)
# a0 -> a
# a1 -> b
# return MCM su a0
mcm:
```

```
mul    s1, a0, a1
jal     ra, mcd
div     a0, s1, a0
```

```
ret
```

Serve salvare qualcosa?

**Simulare questo codice su
RARS**

ra → sovrascritto!



Completare questo esercizio e
consegnarlo su Moodle